Building your own text-generating AI (3/4)

Lecture 10 – Attention is all you need

Dr Thomas Robinson

August 17, 2026

LSE SUMMER SCHOOL · 2026

Warm-up · three questions from last time

≈ 5 MIN · SHOUT IT OUT

1. Which weight matrices does an RNN reuse at every time step?
2. Why do gradients vanish over long sequences in a vanilla RNN?
3. A vanilla RNN has $h_t = \tanh(0.5\, h_{t-1} + 1.0\, x_t)$, with $h_0 = 0$ and $x_1 = 1$. What is h_1?

Warm-up · answers

1. **All of them.** $\mathbf{W}_{hh}$, $\mathbf{W}_{xh}$ and $\mathbf{b}_h$ are the same at every time step. That reuse *is* the parameter sharing — one model, applied T times.
2. **Backprop through time multiplies one step-to-step factor per step** (W_{hh} times a $\tanh'$ term). When those factors sit below 1, the product shrinks towards 0: $0.9^{50} \approx 0.005$.
3. $h_1 = \tanh(0.5 \times 0 + 1.0 \times 1) = \tanh(1) \approx 0.76$.

Part 1 · The attention operation, in three layers

Layer 1 of 3 — the intuition

READING A SENTENCE

> *The keys, which had been sitting on the table next to the mail,* ***were*** *still there.*

To choose the verb form *were* (not *was*), the model has to look back **~12 tokens** — a dozen positions — to find *keys* and ignore *table* and *mail*.

An RNN

Hopes the relevant info survives a dozen hidden-state updates. Usually it doesn't.

Attention

At the position of *were*, **look at every previous token directly** and decide which to focus on.

"Attention" — the network gets to look anywhere it wants, and learns where to look.

In one sentence — what attention does

For each position, take a weighted average of the representations at other positions — where the weights are computed from the data itself.

- “Weighted average”: linear combination.
- “Of representations”: each token’s vector.
- “Weights computed from the data”: the model **learns** which other positions to look at, given the current one.

Note

Notice this is a **soft** version of “select one token”. Instead of picking one, attention takes a **weighted blend** of all of them, with weights summing to 1.

Attention as a soft database lookup

A USEFUL ANALOGY

Imagine a database:

- Each row has a **key** and a **value**.
- You issue a **query**.
- The database returns values whose keys match.

Attention is the **soft** version:

- Every key gets a *similarity score* with the query.
- Return a *weighted average* of all values.
- Weights come from softmax of the similarity scores.

Why "soft"? Because the weights are continuous and differentiable. We can backprop through them.

A hard database lookup is non-differentiable. Attention is what you get when you smooth it out — and the smoothness is also what lets the model *learn* what to attend to.

Layer 2 of 3 — queries, keys, values

For each token position i, produce **three** vectors from its embedding $\mathbf{x}_i$ via learned matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$:

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}_Q, \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}_K, \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}_V$$

MENTAL MODEL

- **Query $\mathbf{q}_i$** — *"what am I looking for?"*
- **Key $\mathbf{k}_j$** — *"what do I offer?"*
- **Value $\mathbf{v}_j$** — *"if you attend to me, here's the content I'll contribute."*

The three are projections of the *same* embedding through *different* learned matrices. So a token can offer different "search terms" than it offers as "search results".

The attention score — how much should i attend to j?

Take the **dot product** of $\mathbf{q}_i$ and $\mathbf{k}_j$.

- Large positive ⇒ $\mathbf{q}_i$ and $\mathbf{k}_j$ point the same way ⇒ i should attend strongly to j.
- Around zero ⇒ no relationship.
- Large negative ⇒ they point opposite ⇒ i should *not* attend to j.

Stack the queries and keys as matrices $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{T \times d}$ (one row per token, T tokens, d dimensions each):

$$\text{scores} = \frac{\mathbf{Q}\mathbf{K}^\mathsf{T}}{\sqrt{d}}, \qquad \text{shape: } (T \times T)$$

Each entry (i, j) = how much i wants to attend to j.

The $\sqrt{d}$ scaling keeps dot products from blowing up in high dimensions.

Strictly, the scale is $\sqrt{d_k}$, the **key dimension** — here a single head uses the full width d, so $d_k = d$. Once we split into h heads (next part), each head works in $d_k = d/h$ and scales by $\sqrt{d/h}$ — exactly what the code does.

Why divide by $\sqrt{d}$? — the problem

We claimed the scaling stops dot products “blowing up”. Let’s put numbers on the blow-up.

Suppose each entry of $\mathbf{q}_i$ and $\mathbf{k}_j$ has mean 0 and variance 1. The dot product adds up d products, one per dimension:

$$\mathbf{q}_i \cdot \mathbf{k}_j = q_{i1}k_{j1} + q_{i2}k_{j2} + \cdots + q_{id}k_{jd}$$

Each term has variance 1. Summing d independent terms gives variance d.

At $d = 64$: variance 64, so standard deviation $\sqrt{64} = 8$.

Raw scores land around ± 8, and gaps of 10 or more between scores are routine. The softmax is about to receive those gaps — the next slide shows what it does with them.

Why divide by $\sqrt{d}$? — what big scores do

Scores $[2, 0, -2]$

$$\text{softmax} = [0.87,\ 0.12,\ 0.02]$$

One position leads. The others keep some weight — the output is still a blend, and every position gets a gradient.

Same scores × 4: $[8, 0, -8]$

$$\text{softmax} = [0.9997,\ 0.0003,\ 0.0000]$$

One position takes everything. The gradients through the near-zero entries die, so the model cannot learn to look elsewhere.

Dividing the scores by $\sqrt{d}$ puts them back at variance 1 — the left-hand column, not the right. The softmax stays spread out, and attention stays trainable.

TRY IT YOURSELF

At $d = 64$, what do we divide the scores by?

ANSWER

$\sqrt{64} = 8.$

Softmax → weights → weighted sum

Take softmax across each row of `scores` — gives a probability distribution over positions:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right), \quad \sum_j A_{ij} = 1$$

WORKED SOFTMAX — ONE ROW OF SCORES $[1, 0, 1]$

$$e^1 = 2.72, \quad e^0 = 1, \quad e^1 = 2.72, \qquad \text{sum} = 6.44$$

$$\left[\tfrac{2.72}{6.44},\ \tfrac{1}{6.44},\ \tfrac{2.72}{6.44}\right] = [0.42,\ 0.16,\ 0.42]$$

The two equal-and-largest scores share most of the weight; the small one keeps a little. The weights sum to 1.

Now use the weights to mix the **values**:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{A}\mathbf{V}$$

For each position i, the output is a **weighted sum** of all value vectors $\mathbf{v}_j$, with weights A_{ij} chosen by the model.

That's the whole attention operation: three matmuls and a softmax.

Self-attention vs cross-attention

SAME OPERATION, TWO DIFFERENT USES

Self-attention

Q, K, V all come from the **same** sequence.

- Used in GPT, Claude, BERT.
- Each token attends to other tokens in the same input.
- This is what we've described so far.

CROSS-ATTENTION

Q comes from one sequence; K, V come from **another**.

- Used in translation (decoder attends to encoder).
- Used in text-to-image diffusion (image attends to text prompt).
- Used in Whisper (decoder attends to audio).

One operation, two roles: looking within yourself (self) or looking at a paired sequence (cross). The arithmetic is identical; what changes is *where you draw Q, K, V from*.

Layer 3 of 3 — causal masking

For language *generation*, position t must not see positions $t+1, t+2, \ldots$ — that would be cheating.

Causal mask: before the softmax, set all entries where $j > i$ to $-\infty$. After softmax, those entries become zero.

The attention matrix becomes **lower-triangular**. Each token attends only to itself and earlier tokens.

Tip

This single line of code turns “see the whole sentence” attention (used in BERT) into “next-token prediction” attention (used in GPT). One change of mask. Same operation otherwise.

Stop and take stock

STOP & CHECK

TRY — 90 SECONDS IN PAIRS

We have $T = 4$ tokens in our sequence. Embedding dimension $d = 8$.

Question: what's the shape of the attention matrix $\mathbf{A}$? And what's the shape of the output of $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$?

ANSWER

$\mathbf{A}$: shape $(T \times T) = (4 \times 4)$.

Attention output: $\mathbf{AV}$, shape $(T \times d) = (4 \times 8)$ — same shape as the original token embeddings.

Attention is shape-preserving. That's important — it means we can stack attention layers without changing the shape.

Plenary · attention by hand

Plenary · attention by hand

≈ 10 MIN · IN PAIRS

A tiny attention example with $T = 3$ tokens and $d = 2$.

$$\mathbf{Q} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 10 & 0 \\ 0 & 10 \\ 5 & 5 \end{bmatrix}$$

(For simplicity, $\mathbf{Q} = \mathbf{K}$ — this is *self-attention*.)

COMPUTE, STEP BY STEP

1. The raw score matrix $\mathbf{Q}\mathbf{K}^\mathsf{T}$ (ignore the $\sqrt{d}$ scaling for now).
2. Apply softmax row-wise (approximate; "the row's biggest entry gets most of the mass").
3. The attention output $\mathbf{A}\mathbf{V}$.
4. **Reflect**: what does token 3 (the row $\begin{bmatrix} 1 & 1 \end{bmatrix}$) attend to most?

Six minutes.

Plenary · answers (1/2)

STEP 1 — SCORES

$$\mathbf{Q}\mathbf{K}^{\mathsf{T}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

STEP 2 — APPROXIMATE ROW-SOFTMAX

Row 1: $[1, 0, 1] \to \approx [0.42, 0.16, 0.42]$. Row 2: $[0, 1, 1] \to \approx [0.16, 0.42, 0.42]$. Row 3: $[1, 1, 2] \to \approx [0.21, 0.21, 0.58]$ (the “2” dominates).

Plenary · answers (2/2)

STEP 3 — ATTENTION OUTPUT $\mathbf{AV}$

Row 3 example:

$$[0.21, 0.21, 0.58] \begin{bmatrix} 10 & 0 \\ 0 & 10 \\ 5 & 5 \end{bmatrix} = [0.21 \cdot 10 + 0.58 \cdot 5,\ 0.21 \cdot 10 + 0.58 \cdot 5] \approx [5.0, 5.0]$$

REFLECTION

Token 3's row is $\begin{bmatrix} 1 & 1 \end{bmatrix}$ — equal in both dimensions. It scores highest with token 3 itself, which has key $\begin{bmatrix} 1 & 1 \end{bmatrix}$.

So **token 3 attends most strongly to itself**. Tokens with directional keys (like $[1, 0]$ or $[0, 1]$) attend most to *matching* directions.

This is the structure attention learns: tokens with similar Q and K vectors find each other.

Part 2 · The transformer block

Multi-head attention

A single attention head learns *one* notion of relevance. We want many.

Multi-head attention: run several attention operations in parallel, each with its own $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$. Concatenate. Project.

For h heads with per-head dim d/h:

$$\mathrm{MHA}(\mathbf{X}) = \mathrm{concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, \mathbf{W}_O$$

Tip

Different heads specialise: * One tracks **coreference** (he/she → person names). * One tracks **syntactic** dependencies. * One tracks **next-likely-word** patterns.

We don't tell them to. They emerge.

Typical $h = 8, 12, 16, 32$.

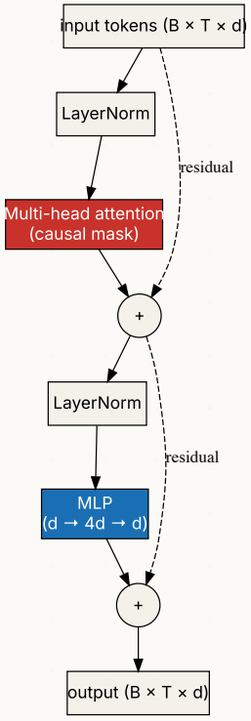
The attention formula in code

FIVE LINES OF PYTORCH

```python
def attention(Q, K, V, mask=None):
    # Q, K, V are (batch, n_heads, seq_len, d_head)
    scores = Q @ K.transpose(-2, -1)          # (B, h, T, T)
    scores = scores / math.sqrt(Q.size(-1))  # scale by √d
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)
    weights = F.softmax(scores, dim=-1)       # (B, h, T, T)
    return weights @ V                        # (B, h, T, d_head)
```

Every major transformer — GPT, Claude, BERT — runs some version of these five lines. The rest is plumbing — splitting heads, applying the mask, projecting the output. The core computation is what you see here.

One transformer block



Each transformer block is: LayerNorm → attention → residual; then LayerNorm → MLP → residual.

Repeat this block N times. **That's a transformer.**

Residual connections — why they matter

THE DASHED LINES IN THE DIAGRAM

Each sublayer's output is **added** to its input:

$$\mathbf{x}_{\text{out}} = \mathbf{x}_{\text{in}} + f(\mathbf{x}_{\text{in}})$$

- Sublayer learns a **correction** to the input, not the full transformation.
- Differentiating $\mathbf{x}_{\text{in}} + f(\mathbf{x}_{\text{in}})$ gives a **+1** term from the identity path, so the upstream gradient can always flow straight back along that path — it never fully vanishes.
- Stacks of ResNet-style residuals are **the most reliable way** to train networks deeper than ~10 layers.

Note

First introduced for image CNNs (He et al. 2015 — ResNet). Transformers inherited them. Without residuals, a 100-layer transformer would not train.

LayerNorm — the other half

WHY BEFORE VS AFTER THE SUBLAYER MATTERS

LayerNorm normalises each token's vector to zero mean, unit variance, then applies a learned scale and shift.

- Original transformer: LN *after* the sublayer ("Post-LN").
- Modern transformers: LN *before* ("Pre-LN", what we drew earlier).

The order matters for training stability — Pre-LN trains more reliably, especially at large scale.

Why it works. Activations across layers stay roughly the same scale, so gradients propagate through dozens of blocks. Combined with residuals, you get an architecture that's robust to depth.

Skipping LayerNorm in a 50-layer transformer turns it into a black hole of NaN loss.

Why each piece is there

Multi-head attention
: Lets every position look at every other (with causal mask).

MLP (a small feed-forward block)
: Position-wise non-linear processing. Standard 2-layer FNN with GELU activation.

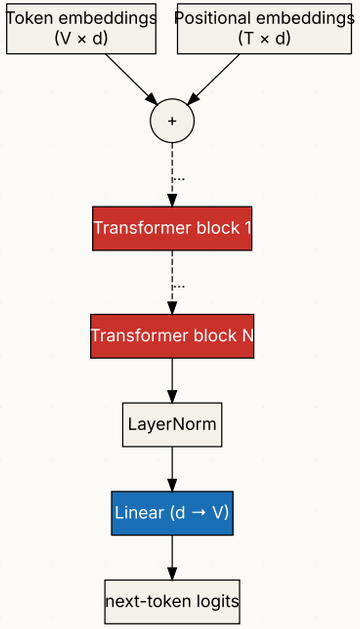
LayerNorm
: Normalises activations per-token. Stabilises training of deep stacks.

Residual connections
: Each sublayer's output is **added** to its input. The identity path lets gradients flow through 100+ layers without vanishing (we saw the +1 trick two slides back).

Note

Four pieces, four jobs: **attend** (look around), **process** (MLP), **normalise** (LayerNorm), **preserve the gradient** (residual). Keep this map — the Activity later asks you to match plain-English descriptions back to each one.

The full GPT architecture



- Token embedding **adds** positional embedding — attention is permutation-invariant otherwise.
- N identical transformer blocks.
- Final LayerNorm, then a linear projection back to vocabulary size.

Counting GPT-2's parameters — the embeddings

We have counted the parameters of every architecture in this course, biases included. The transformer's turn.

GPT-2 SMALL — THE SPECIFICATION

Vocabulary $V = 50{,}257$ · context $T = 1{,}024$ · width $d = 768$ · $N = 12$ blocks · $h = 12$ heads.

THE TWO EMBEDDING TABLES

- **Token embeddings** — one d-vector per vocabulary entry: $50{,}257 \times 768 = 38{,}597{,}376$
- **Positional embeddings** — one d-vector per position: $1{,}024 \times 768 = 786{,}432$

Running total: $39{,}383{,}808$.

Lookup tables have no biases — nothing gets multiplied, so there is nothing to shift.

Counting GPT-2's parameters — one block

Every linear layer costs (inputs × outputs) + outputs. Biases included, as always.

Attention

- QKV projections, $d \to 3d$: $768 \times 2{,}304 + 2{,}304 = 1{,}771{,}776$
- Output projection $\mathbf{W}_O$, $d \to d$: $768 \times 768 + 768 = 590{,}592$

MLP

- Up, $d \to 4d$: $768 \times 3{,}072 + 3{,}072 = 2{,}362{,}368$
- Down, $4d \to d$: $3{,}072 \times 768 + 768 = 2{,}360{,}064$

Two LayerNorms, each a learned scale and shift per dimension: $2 \times 2 \times 768 = 3{,}072$.

One block: $1{,}771{,}776 + 590{,}592 + 2{,}362{,}368 + 2{,}360{,}064 + 3{,}072 = 7{,}087{,}872$.

Twelve blocks: $12 \times 7{,}087{,}872 = 85{,}054{,}464$.

Counting GPT-2's parameters — the total

Two pieces left after the blocks:

- Final LayerNorm: $2 \times 768 = 1{,}536$.
- The $d \to V$ head **reuses the token embedding table**, transposed — this is **weight tying**. Cost: 0 new parameters.

$$39{,}383{,}808 + 85{,}054{,}464 + 1{,}536 = 124{,}439{,}808 \approx 124\text{M}$$

The famous number. When you read "GPT-2 small, 124M parameters", you can now account for every one of them.

WHERE THE PARAMETERS LIVE

Component	Count	Share
Embeddings	$39{,}383{,}808$	~32%
MLPs	$56{,}669{,}184$	~46%
Attention	$28{,}348{,}416$	~23%
LayerNorms	$38{,}400$	~0.03%

Attention gets the headlines; the MLPs hold nearly half the parameters.

What’s the positional embedding doing?

AN ASIDE THAT CONFUSES STUDENTS

Without positional information, attention is **permutation-invariant** — it would give the same output for `"dog bites man"` and `"man bites dog"`.

That’s wrong. So we **add** a vector that encodes *which position* each token is in.

There are three common ways to build that position vector — **sinusoidal**, **learned**, and **RoPE** — which we compare next. The details matter for engineering, not for the core idea: *give every position a fingerprint the model can read.*

Positional encoding — three variants

HOW TO BUILD THE POSITION FINGERPRINT

Sinusoidal

Original transformer (2017). Deterministic, no learnable parameters. Embeds position t as a vector of sines and cosines at different frequencies.

Learned

One parameter vector per position, learned alongside the model. Used in BERT, GPT-2/3.

Limitation: can't extrapolate beyond training-time length.

RoPE (rotary)

Rotates Q and K vectors by an angle proportional to position. Modern default — Llama, Mistral, Qwen.

Extrapolates more gracefully to longer contexts.

Why attention is O(T²)

THE DEFINING COST — AND THE LIMIT IT IMPOSES

The attention matrix is $T \times T$.

For $T = 1{,}000$: 1M entries. Manageable. For $T = 10{,}000$: 100M. Painful. For $T = 1{,}000{,}000$: a *trillion* entries.

Compute cost scales as $T^2 \cdot d$ per layer. Memory scales as T^2.

This is the dominant constraint on context length. A 1M-token GPT call is technically possible but expensive.

Architectures like **Mamba**, **RingAttention**, and **Mixture-of-Depths** attack the $O(T^2)$ cost in different ways. (**FlashAttention** is often lumped in here, but it computes *exact* attention — still $O(T^2)$ FLOPs, just with less memory traffic.)

Training and inference — the contrast

Training

- Take a long sequence (say, 1024 tokens).
- Each position predicts the **next** token.
- Cross-entropy loss summed across all positions.
- All positions trained **in parallel** in one forward pass.

This parallelism is **why transformers won** over RNNs.

INFERENCE

- Encode the prompt.
- Compute logits at the last position.
- Sample (or argmax) the next token.
- Append. Repeat.

Auto-regressive: one token at a time. The model never sees the next token before it predicts it.

> **Note**
>
> ChatGPT's response feels live because it's generating **one token at a time** in real time. Modern infrastructure can do this hundreds of times per second for tens of thousands of users simultaneously.

The KV cache — don’t recompute what hasn’t changed

The inference loop runs the model once per token. Most of that work is identical from step to step.

When we generate token $T+1$, the keys and values of tokens $1 \dots T$ **have not changed** — same tokens, same weights, same $\mathbf{k}_j$ and $\mathbf{v}_j$ as the last step.

The **KV cache**: keep every layer’s keys and values in GPU memory. Each step then

1. computes one new query, key and value — for the new token only;
2. scores the new query against the cached keys;
3. appends the new key and value to the cache.

Without the cache

Every new token recomputes attention for the whole prefix — all T queries, keys and values, at every layer, every step.

With the cache

Every new token computes one row of the attention matrix — one query against T stored keys.

The KV cache — the memory bill

The cache trades compute for memory. How much memory?

PER TOKEN CACHED — GPT-2 SMALL, FP16

$$\underbrace{12}_{\text{layers}} \times \underbrace{2}_{K \text{ and } V} \times \underbrace{768}_{\text{width } d} \times \underbrace{2}_{\text{bytes (fp16)}} = 36{,}864 \text{ bytes} \approx 36 \text{ KB}$$

A 1,000-TOKEN CONVERSATION

$$1{,}000 \times 36{,}864 = 36{,}864{,}000 \text{ bytes} \approx 37 \text{ MB per sequence}$$

Each user's chat carries its own cache. GPT-2 small's weights in fp16 come to $124{,}439{,}808 \times 2 \approx 249$ MB — seven of these conversations match the whole model. Serve a few hundred long chats at once and the caches, not the weights, are what fill GPU memory.

Note

This is the $\mathcal{O}(T^2)$ cost from earlier, paid at inference one row at a time: step t does $\mathcal{O}(t)$ work against the cache, and $1 + 2 + \cdots + T$ still sums to $\mathcal{O}(T^2)$.

Activity · spot each component

Activity · spot the role

≈ 8 MIN · IN PAIRS

FOR EACH OF THESE STATEMENTS, NAME WHICH COMPONENT DOES IT

1. “Lets a token look at every previous token directly.”
2. “Adds non-linear processing per position.”
3. “Stabilises activation distributions across tokens within a block.”
4. “Lets gradients flow back through 100 layers without vanishing.”
5. “Tells the model which position each token is at.”
6. “Turns the final hidden vector into a distribution over the vocabulary.”
7. “Prevents a token from cheating by looking at the future.”
8. “Lets different attention computations focus on different aspects.”

Discuss in pairs. Then we’ll go through them.

Activity · answers

1. **Self-attention.**
2. **The MLP (feed-forward) sub-block.**
3. **LayerNorm.**
4. **Residual connections.**
5. **Positional embedding.**
6. **The final linear layer (the “head”) + softmax.**
7. **The causal mask.**
8. **Multi-head attention.**

Tip

That’s the whole transformer accounted for. Every component does one specific job — and you can name what each one is for.

When you read transformer code tomorrow, **this list is your map.**

Part 3 · Scaling

What changed as the models got bigger?

Model	Params	Year	Train compute
GPT-1	117M	2018	tiny
GPT-2	1.5B	2019	~1 GPU-month
GPT-3	175B	2020	~3 GWh
GPT-4	~1.8T (estim.)	2023	tens of GWh
Claude 4.7 Opus	undisclosed	2026	undisclosed

The core block design is the same across these models. What changes:

- Width, depth, head count.
- Data: more tokens, better filtering.
- Training tricks: warmup, learning-rate schedules, fp16/bf16.
- **Post-training**: instruction tuning, RLHF — we don't cover this in detail.

Scaling laws

KAPLAN ET AL. (2020), HOFFMANN ET AL. (2022)

Empirical regularity: model loss as a function of compute, parameters, and training tokens follows a **power law**.

- Doubling parameters → ~constant fractional drop in loss.
- Same for doubling data, or doubling compute.
- Extends over many orders of magnitude.

Chinchilla (2022): the "optimal" allocation is roughly **20 tokens per parameter** — the previous era over-parameterised and under-trained.

The reason every lab now trains for as long as they can afford. The reason "more data + more compute" is the answer to most questions.

The reason **the cost of being at the frontier doubles every ~6 months** — and the reason there are now only ~5 organisations on the frontier.

Your turn — Chinchilla maths

≈ 3 MIN · IN PAIRS

TRY

You're training a **1-billion-parameter** model. Chinchilla says compute-optimal is **~20 tokens per parameter**.

1. How many training tokens should you aim for?
2. Your budget only stretches to **5 billion** tokens. Over- or under-parameterised — and what would you change, the model or the data?

ANSWER

1. $20 \times 1{,}000{,}000{,}000 = 20$ **billion tokens**. 2. With only 5B tokens you'd feed a 1B model a quarter of its optimal data — **under-trained / over-parameterised**. The Chinchilla fix: **shrink the model** to ~250M params ($5\text{B} \div 20$) so it matches the data, *or* go find 15B more tokens.

“Emergent” capabilities

A surprise of the GPT-3 era:

> *Capabilities like multi-step arithmetic, code synthesis, and translation* ***don’t gradually improve*** *with scale — they appear roughly step-wise once a model gets big enough.*

Warning

Active debate over whether “emergence” is real or a measurement artefact (Schaeffer et al. 2023). Either way, the practical consequence is the same: **scale unlocks capabilities you didn’t know to test for**.

For policy and ethics, this matters: the model that ships next year may have abilities the model in your lab doesn’t.

Mixture of experts — the inference trick

A WAY TO GROW PARAMETER COUNT WITHOUT GROWING INFERENCE COST

The idea: replace the dense feed-forward sublayer with a *bank* of feed-forward "experts". A small router network picks ~2 of them per token.

- Total params: huge (8× or 16× a dense model).
- Active params *per token*: only the picked experts.
- Inference cost: stays low.

Used by:

- **Mixtral 8×7B** (Mistral, 2023).
- **GPT-4** (rumoured to be MoE).
- **DeepSeek V3** and Gemini.

The 2024+ frontier is increasingly MoE. The same architecture-stacking principle — but with sparsity along the depth axis.

Limitations even at scale

Computational

- The $\mathcal{O}(T^2)$ attention cost (Part 2) still caps practical context length.
- Generation is **sequential** — one token at a time, so latency grows with output length.
- Inference cost is real — every API call burns energy.

Conceptual

- Hallucination is **built into the architecture**.
- No internal "truth" signal.
- Brittle reasoning under distribution shift.
- Encodes biases of training data.

We come back to ethics on Day 12.

A connection to social science — LLMs as instruments

A live research question: can LLMs be used as **measurement instruments** for social-scientific constructs?

- Argyle et al. (2023): can an LLM "simulate" a respondent of a given demographic? (Partially, with caveats.)
- Spirling (2023): argues social science should rely on **open** models, not closed APIs, so that text-based measurements stay reproducible. (A live methodological debate.)
- Bender et al. (2021) — *Stochastic Parrots* — raises the deeper question of whether asking these models for **meaning** at all is a category error.

Tip

These debates are about **the exact architecture you'll have built by today's lab**. You're now equipped to read them critically.

Take stock

WHERE YOU ARE AT THE END OF DAY 10

You can:

1. State, in one sentence, what attention does.
2. Compute a small attention operation by hand.
3. Name every component of a transformer block.
4. Explain why residuals + LayerNorm enable deep stacks.
5. Describe the difference between training (parallel) and inference (autoregressive).
6. Articulate at least one debate over what LLMs *mean* for social science.

This is the architecture behind GPT, Claude, and every modern LLM.

That was three weeks ago a black box. Today: a thing you can read.

Show your GPT

AFTER TODAY'S LAB — BRING TOMORROW

AT THE END OF TODAY'S LAB

Your tiny GPT will generate some text. Pick **the funniest sentence it produces** — or the one that almost makes sense — and bring it tomorrow.

We'll spend ten minutes at the start of Day 11 reading the best ones aloud. **Show off what you built.**

Tip

The output should still be *not very good* — it's a tiny model on a small corpus. But every now and then, the model produces something that's structurally English, grammatically right, and almost makes sense. Those are the moments worth sharing.

Lab brief · today’s afternoon

Today’s lab — 90 minutes

GOALS

Build a **tiny GPT** end-to-end. Same character corpus as Thursday’s RNN. Compare results.

You will implement:

1. Token + positional embeddings.
2. **Causal multi-head self-attention.**
3. The two-sublayer transformer block (attention + MLP + residuals + LayerNorm).
4. A stack of $N = 4$ such blocks.
5. The training loop and a sampler.

`nanoGPT` BY KARPATHY IS THE CANONICAL REFERENCE

Read it **after** the lab — every design decision in modern LLMs is in there in ~300 lines of code. Your lab implementation mirrors it.

Stretch: vary the number of layers N from 1 to 6. Plot val loss. Where does the diminishing return kick in?

See you tomorrow

Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? *Proceedings of FAccT*.